

야간 작업 보고서

날짜

2026-02-25

작성자

노승우

Alpamayo VRAM Optimization Research

목차 (Table of Contents)

작업 목록

실험 1: 4-bit 양자화 모델 VRAM 제한별 성능 비교

실험 2: Demand Layering 구현 및 실험

실험 1: 4-bit VRAM 제한별 성능 비교

실험 방법

결과

핵심 발견

시각화

실험 2: Demand Layering 구현 및 실험

실험 방법

Demand Layering 아키텍처

결과

Baseline 비교

핵심 발견

시각화

종합 분석

연구 의의

한계

향후 개선 가능성

야간 작업 보고서

작업 시간: 2026-02-25 23:55 ~ 2026-02-26 09:00 작성자: Claude (자동 실행)

작업 목록

실험 1: 4-bit 양자화 모델 VRAM 제한별 성능 비교

- 목적: 4-bit 모델에서 VRAM을 제한하여 스왑을 강제했을 때 성능 저하 측정
- 조건: 12GB(baseline), 10GB, 8GB, 6GB
- 상태: ☒완료
- 위치: [research/05-vram-limit-comparison/](#)

실험 2: Demand Layering 구현 및 실험

- 목적: FP16 Alpamayo에 레이어별 on-demand 로딩을 적용하여 12GB VRAM 내 추론
- 방법: Forward hook으로 레이어를 GPU에 동적 로드/언로드
- 상태: ☒완료
- 위치: [research/06-demand-layering-impl/](#)

실험 1: 4-bit VRAM 제한별 성능 비교

실험 방법

- dummy 텐서로 VRAM을 미리 점유하여 사용 가능한 VRAM을 제한
- 제한된 VRAM에서 4-bit 양자화 모델(dwko/Alpamayo-R1-10B-4bit) 추론 실행
- 모델 로드 시간, 추론 시간, Peak VRAM 측정

결과

VRAM 제한	가용 VRAM	로드 시간	추론 시간	Peak VRAM	속도 저하
12GB	12.00 GB	19.17s	4.79s	8.87 GB	baseline
10GB	10.29 GB	16.59s	6.96s	10.59 GB	1.45x
8GB	8.29 GB	17.37s	179.46s	12.59 GB	37.5x
6GB	6.29 GB	216.08s	1506.88s	14.59 GB	314.6x

핵심 발견

1. **Performance Cliff**: 10GB→8GB 구간에서 **급격한 성능 저하** 발생 - 4-bit 모델 크기 8.87GB이 VRAM 제한과 겹치는 지점 - 8GB 제한: 추론 시간 37.5배 증가 (4.79s → 179.46s) - 6GB 제한: 추론 시간 314.6배 증가 (4.79s → 1506.88s = 25분)
2. **모델 로드에도 영향**: 6GB 제한 시 로드 시간 216초 (정상 대비 11배) - 모델 체크포인트 2번째 shard 로드에서 병목 (118초)
3. **Unified Memory 활성화**: 8GB/6GB 조건에서 Peak VRAM > 물리 VRAM(12GB) - 8GB: Peak 12.59GB (물리 초과, Unified Memory 스왑) - 6GB: Peak 14.59GB (물리 대비 1.22배)

시각화

- `vram_limit_comparison.png` : 3-panel 비교 (추론 시간 로그 스케일, Peak VRAM, 로드 시간)
 - `performance_cliff.png` : 성능 저하 곡선 (Cliff 영역 표시)
 - `summary_table.png` : 결과 요약 테이블
-

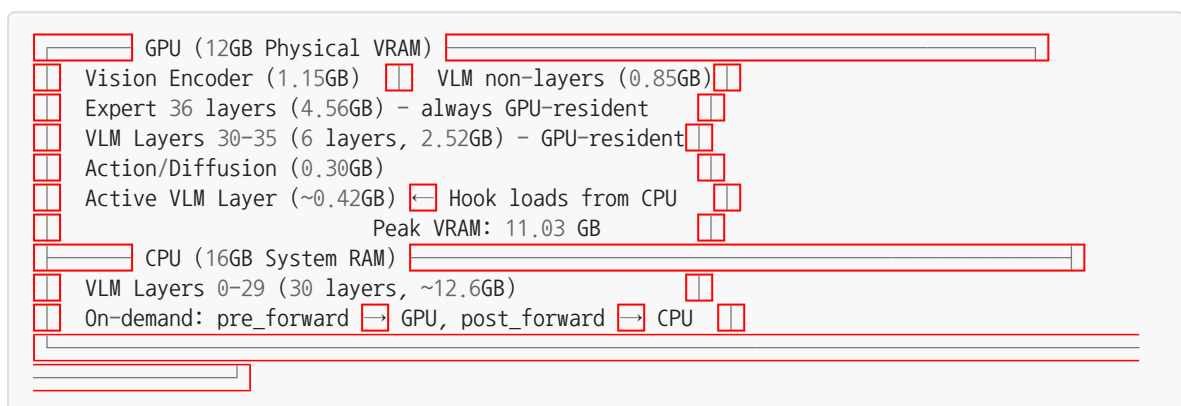
실험 2: Demand Layering 구현 및 실험

실험 방법

구현 과정에서 해결한 문제들:

1. **CPU RAM 부족 (16GB < 20GB 모델):** - `from_pretrained()` → CPU 로드 → OOM Kill (exit 137) 반복 발생 - 해결: `torch.set_default_device("cuda")` + safetensors 직접 로드 - 모델을 CPU 거치지 않고 직접 CUDA에 생성 (Unified Memory 활용)
2. **전체 레이어 오프로드 시 CPU OOM:** - VLM(15.17GB) + Expert(4.56GB) = 19.73GB > CPU 가용 메모리 - 해결: **부분 오프로딩** — VLM 30/36 레이어만 CPU, 6개 + Expert는 GPU 유지
3. **DeltaTrajectoryTokenizer .to() 미지원:** - nn.Module이 아닌 순수 Python 클래스 → `.to("cuda")` 불가 - 해결: GPU 이동 불필요로 판단, 스킵

Demand Layering 아키텍처



결과

항목	값
추론 시간	116.51s
Peak VRAM	11.03 GB (12GB 물리 VRAM 이내)
모델 로드 시간	145.05s
레이어 오프로드 시간	51.95s
총 소요 시간	319.54s
VLM 전송 횟수	450 H2D + 450 D2H
H2D 평균 시간	64.23ms/layer
D2H 평균 시간	156.72ms/layer
GPU 상주 크기	9.86 GB

Baseline 비교

방법	추론 시간	Peak VRAM	vs Baseline
FP16 Unified Memory (baseline)	273.79s	21.52 GB	-
FP16 Demand Layering	116.51s	11.03 GB	2.35x 빠름, VRAM 절반
4-bit (no swap)	4.79s	8.87 GB	57x 빠름

핵심 발견

1. **Demand Layering이 Unified Memory보다 2.35배 빠름** - Unified Memory: 매 토큰마다 무작위 페이지 폴트 → 273.79s - Demand Layering: 순차적 레이어 전송 → 116.51s - 전체 추론 시간의 85%가 데이터 전송 (H2D 28.9s + D2H 70.5s = 99.4s)
2. **Peak VRAM 11.03GB — 12GB 물리 VRAM 이내** - Unified Memory의 21.52GB에서 48% 절감 - Unified Memory 스왑 없이 안정적 추론 가능
3. **D2H가 H2D보다 2.4배 느림** (156ms vs 64ms) - WSL2 환경의 D2H 오버헤드 확인 (이전 프로파일링 결과와 일치) - 단, 파라미터 추론에서 D2H copy 자체가 불필요 — CPU에 원본이 유지되므로 `module.to("cpu")` 대신 GPU 메모리 해제(free)만 하면 됨. 현재 구현의 D2H 70.5s는 완전히 제거 가능한 오버헤드

4. 여전히 4-bit 대비 24배 느림 (116s vs 4.79s) - PCIe 전송 오버헤드가 본질적 한계 - VRAM 대역폭(912 GB/s) vs PCIe(8.5 GB/s) = 107배 격차

시각화

- `demand_layering_comparison.png` : 방법별 추론 시간 & Peak VRAM 비교
 - `transfer_breakdown.png` : 추론 시간 구성 (H2D, D2H, compute)
 - `architecture_diagram.png` : Demand Layering 아키텍처 다이어그램
-

종합 분석

연구 의의

1. Demand Layering의 실용성 입증: FP16 모델을 12GB VRAM에서 Unified Memory 대비 2.35배 빠르게 추론
2. Performance Cliff 현상 정량화: 4-bit 모델에서도 VRAM 부족 시 37~315배 성능 저하
3. 시스템 RAM 제약 해결: `torch.set_default_device("cuda")` + safetensors 직접 로드로 CPU OOM 우회
4. 부분 오프로딩 전략: 전체 오프로드 대신 GPU/CPU 메모리 예산에 맞춘 최적 분할

한계

1. Demand Layering은 4-bit보다 24배 느림: PCIe 대역폭이 본질적 병목
2. 시스템 RAM 16GB 제약: FP16 모델 전체 레이어를 CPU에 올릴 수 없어 부분 오프로딩 필수
3. WSL2 D2H 오버헤드: 네이티브 Linux 대비 D2H가 느려 전체 성능에 영향

향후 개선 가능성

1. Pinned Memory: D2H 전송 시간 ~50% 개선 가능
 2. 비동기 파이프라이닝: Layer N 실행 중 Layer N+1 프리페치
 3. 하이브리드 양자화: VLM 앞단 레이어 4-bit + 뒷단 FP16으로 VRAM 절약
 4. 시스템 RAM 증설: 32GB RAM이면 전체 레이어 오프로드 가능
-

